AI ML ASSIGNMENT -BRAINBOT

WEEK 3 REPORT

Development & Implementation:

The main objective for Week 3 was to develop the core functionality of the Bot brain prototype. This involved building the chatbot's interface, integrating natural language processing (NLP) to understand user requests, and implementing the navigation and data modules.

1. Chatbot Interface & NLP

- What I did: My initial plan was to use Google Dialogflow for the chatbot's conversational component. As a platform, Dialogflow is excellent for natural language understanding and managing conversation flow.

  However, I encountered a significant technical challenge during the implementation phase: integrating the pathfinding algorithms with Dialogflow.It's is a cloud service, and to connect it to my custom Python code, I would have needed to set up a fulfillment webhook. This would have required me toDeploy my pathfinding code to a cloud service (like Google Cloud Functions or another web server).

- So, I developed a custom chatbot interface using Python with the Flask framework and a single HTML file.

  This allowed for full control over the look and feel. The front end is a chat window.

- **How I met the requirement:** I implemented a **rule-based NLP system** to handle user queries. This system, located in the extract_locations function, uses regular expressions and string matching to identify keywords like "from," "to," and specific location names. It's a fundamental approach that effectively extracts the user's intent to find a route.

2. Navigation Module

- **What I did:** I built a comprehensive navigation module based on a **graph data structure**. The campus locations are represented as nodes, and the paths between them are edges with weights equal to their real-world distances.
- **How I met the requirement:** I implemented four different pathfinding algorithms: **Breadth-First Search (BFS), Depth-First Search (DFS), Uniform-**

Cost Search (UCS), and A* (A-star) Search. The system uses the haversine formula to calculate accurate distances between the GPS coordinates of each location. This is the core of the navigation feature.

3. Database Connection

- **What I did:** I simulated a database by using Python dictionaries (dms_coordinates and decimal_coordinates) to store all the campus data. This approach demonstrates the ability to connect to and retrieve information from a data source.

```
# GPS coordinates in DMS format
dms_coordinates = {
    "Main Gate": ("13°13'16.7\"N", "77°45'18.2\"E"),
    "Admin Block": ("13°13'19.9\"N", "77°45'18.9\"E"),
    "Academic Block 1": ("13°13'24.0\"N", "77°45'17.7\"E"),
    "Academic Block 2": ("13°13'24.2\"N", "77°45'21.4\"E"),
    "Academic Block 3": ("13°13'20.6\"N", "77°45'22.6\"E"),
    "Hostel": ("13°13'28.3\"N", "77°45'32.6\"E"),
    "Food Court": ("13°13'29.5\"N", "77°45'26.0\"E"),
    "Sports Complex": ("13°13'42.3\"N", "77°45'29.8\"E"),
    "Central Junction": ("13°13'22.0\"N", "77°45'20.2\"E"),
    "Laundry": ("13°13'28.3\"N", "77°45'25.4\"E"),
    "Mini Mart": ("13°13'28.4\"N", "77°45'32.9\"E")
}
```

```
# Convert all coordinates to decimal degrees
decimal_coordinates = {}
for location, (lat_dms, lon_dms) in dms_coordinates.items():
    lat_deg, lat_min, lat_sec, lat_dir = int(lat_dms.split("°")[0]), int(lat_dms.split("°")[1].split
    lon_deg, lon_min, lon_sec, lon_dir = int(lon_dms.split("°")[0]), int(lon_dms.split("°")[1].split
    lat_decimal = dms_to_decimal(lat_deg, lat_min, lat_sec, lat_dir)
    lon_decimal = dms_to_decimal(lon_deg, lon_min, lon_sec, lon_dir)
    decimal_coordinates[location] = (lat_decimal, lon_decimal)
```

- **How I met the requirement:** The chatbot can access this "in-memory database" to look up GPS coordinates, check for valid locations, and build the navigation graph.

Deliverables

- **Prototype Chatbot:** The Flask application provides a fully working chat interface that can handle location queries like "Go from Hostel to Academic Block 1.",etc. It provides a conversational response and a visual route.
- **Working Database Connection:** The system successfully retrieves and uses location data from the defined Python dictionary, fulfilling the requirement of having a data source.
- **Navigation Integration:** The chatbot seamlessly uses the integrated pathfinding algorithms to find and display optimal routes, including a static Google Maps image of the path.